

WIRELESS NETWORK AND SECURITY

Lecture 14: NFC Security

Instructor: Jinsong Han

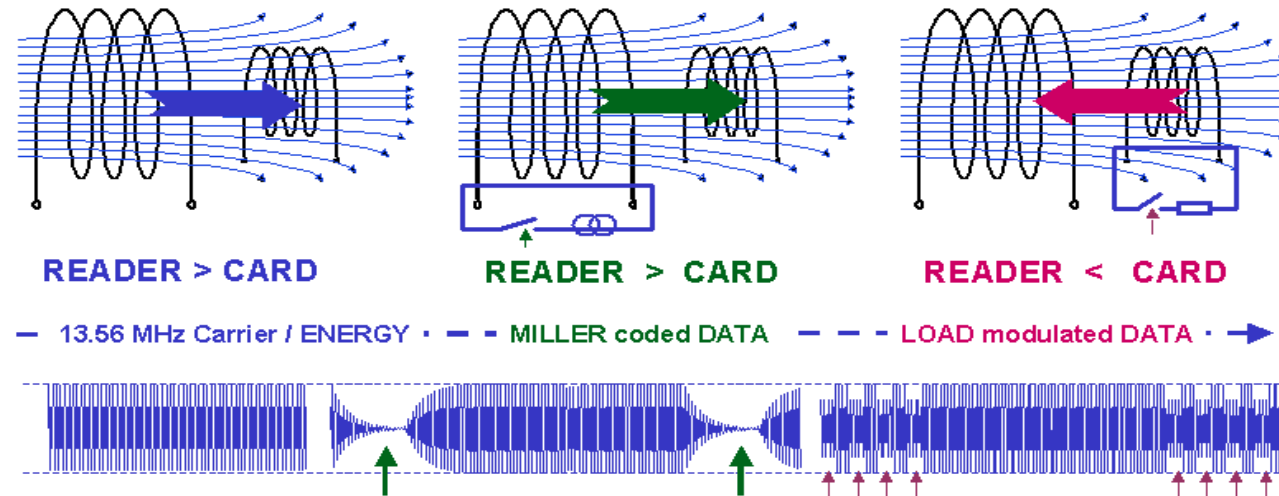


Near Field Communication (NFC)

- Set of short-range, high-frequency Radio Frequency Identity (RFID) data transmission standards
 - ISO 14443, ISO 15693, ISO 18092
- Operating within 4 cm to 10 cm at 13.56 MHz
- Data rates “of NFC radio”: 106 kbit/s, 212 kbit/s, 424 kbit/s
- Communication between two devices:
 - Initiator: E.g. Reader
 - Target: E.g. Tag
- NFC Forum defines:
 - Interoperability
 - NFC application specification



NFC data exchange principle



- Active Device (reader)
 - Proximity coupling device (PCD)
 - Connected to power source
 - Generates an electromagnetic field for data exchange
- Passive device (NFC tag)
 - Proximity integrated circuit card (PICC)
 - Harvest power from an Active device

Active NFC device modes of operation

- Reader / Writer Mode (PCD, ISO 14443)
 - Active device that transmits power
 - Reads and modifies data stored in a passive tag
 - E.g. Mobile phone reading smart poster
- Card Emulation Mode (PICC, ISO 14443)
 - Acts like a passive target
 - Interacts with an external active reader
 - E.g. Mobile phone used as a transport ticket, Google Wallet
- Peer-to-peer Mode (ISO 18092)
 - Both the initiator and target transmit power
 - Establishes bi-directional data connection
 - E.g. A mobile phone exchanging a virtual business card with another mobile phone, Android Beam





NFC forum tag types

	Type 1 Tags	Type 2 Tags	Type 3 Tags	Type 4 Tags
Unique Identity	4 or 7 bytes	4 or 7 bytes	8 bytes	7 bytes
Transmission Protocol	ISO 14443A	ISO 14443A	ISO 18092	ISO 14443A
Memory Size	96 bytes (up to 2 KB)	64 bytes (up to 2 KB)	Variable sizes (up to 1 MB)	Variable sizes (up to 32KB)
Memory Organization	12 blocks, each of 8 bytes	16 pages, each of 4 bytes	Blocks, each of 16 bytes	Smart card based.
OTP bits	48 bits	32 bits		
Lock bits	16 bits	16/32 bits		
Re-writable	Until locked	Until locked	Pre-defined	Issuer-defined
Data collision protection	No	Yes	Yes	Yes
Transmission speed	106 kbits/s	106 kbits/s	212 kbits/s or 424 kbits/s	106 kbits/s, 212 kbits/s, 424 kbits/s
Examples	Topaz	Ultralight	FeliCa Lite	Java cards, DESFire

Threats on memory tags

- **Tag cloning**
 - E.g. Foursquare check-in tags can be cloned to falsely claim that you have been at the location (to claim loyalty discounts)
 - Can be prevented to some degree by calculating MAC that includes UID.
- **Modification of tag data**
 - Can be prevented by locking tag re-write
- **Swapping / replacing valid tags**
 - E.g. Tags used to help purchase items from vending machines can be swapped so that when a customer tries to purchase an item from a vending machine, the immoral person waiting at the other vending machine gets the purchased item.

Example: Security on Type 2 tag (MIFARE Ultralight)

- Byte 0 – 9 : read only
- Byte 10 – 15: One time programmable (OTP) bytes; Once a bit in an OTP byte is set, it cannot be reset back.
- L : Lock page
- BL: Block lock, Once a BL bit is set the locking configuration for the corresponding page is unchangeable.

Page address	Byte Numbers			
00h	UID0	UID1	UID2	BCC0
01h	UID3	UID4	UID5	UID6
02h	BICC1	INT	LOCK0	LOCK1
03h	OTP0	OTP1	OTP2	OTP3
04h	Data0	Data1	Data2	Data3
05h	Data4	Data5	Data6	Data7
....				
0Fh	Data44	Data45	Data46	Data47

Bits	7	6	5	4	3	2	1	0
LOCK 0	L7	L6	L5	L4	L3	BL 15-10	BL 9-4	BL OTP
LOCK 1	L15	L14	L13	L12	L11	L10	L9	L8

NFC tag security

- Security mechanisms:
 - 7-byte Unique Identity (UID)
 - One-time programmable bits: bits that can be set to one but not reset to zero
can be used as counter
 - Pages can be **locked** to prevent modification
- Security assumptions:
 - The UID cannot be cloned or spoofed **!!!?**
 - When reading the tag, the UID and card content cannot be modified by the attacker (physical session integrity) **!!!?**
 - Attacker can freely write data to the card, and interrupt sessions (tear card away from the reader)

Cloning Ultralight tags

- Clonable cards are available
 - Rewrite the entire memory area including UID, OTP and Lock bytes
- Demo: Ultralight Tag cloning

<https://haokan.baidu.com/v?pd=wisenatural&vid=10765615942419811107>

<https://www.youtube.com/watch?v=nn-nuhAByRk>

<https://jerrygamblin.com/2015/12/02/cloning-uids-with-chameleon/>

Ultralight C

- Memory organization is similar to Ultralight
- More memory 192 bytes
- Additional 32-bit one way counter
- Access control using an **authentication key**
 - Write protected or both read/write protected

No Cryptographic security included in the NFC Specs

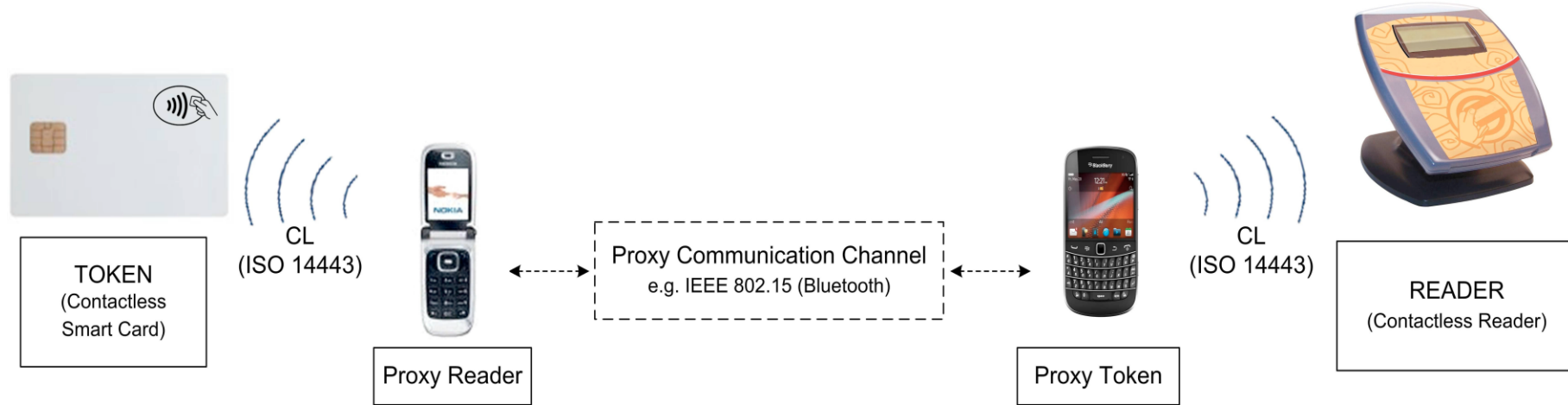
- NFC transmission protocols do not define any specific encryption or security mechanism
 - ISO 14443 : Read/Write and Card Emulation mode
 - ISO 18092: Peer-to-peer mode
- NDEF specification defines signature scheme for integrity protection
 - Does not prevent content cloning (signature does not cover the card UID)
 - Does not include reader authentication for access control
- Therefore, cryptographic security must be defined by the NFC application.

Contactless smart cards

- **Memory tags** with some security functionality
 - MIFARE Ultralight: UID, lock bytes, OTP
 - Ultralight C: **triple-DES authentication**
 - DESFire: **Triple-DES / AES mutual authentication**, file system with access control lists
- **Smart cards** with contactless interface
 - CPU and operating system
 - Tamper-resistant processing environment
 - Secure crypto-processor
 - Secure file system
 - E.g. JavaCard, EMV contactless debit and credit cards
- *Distinction between memory cards and smart cards is not always clear cut*₁₂



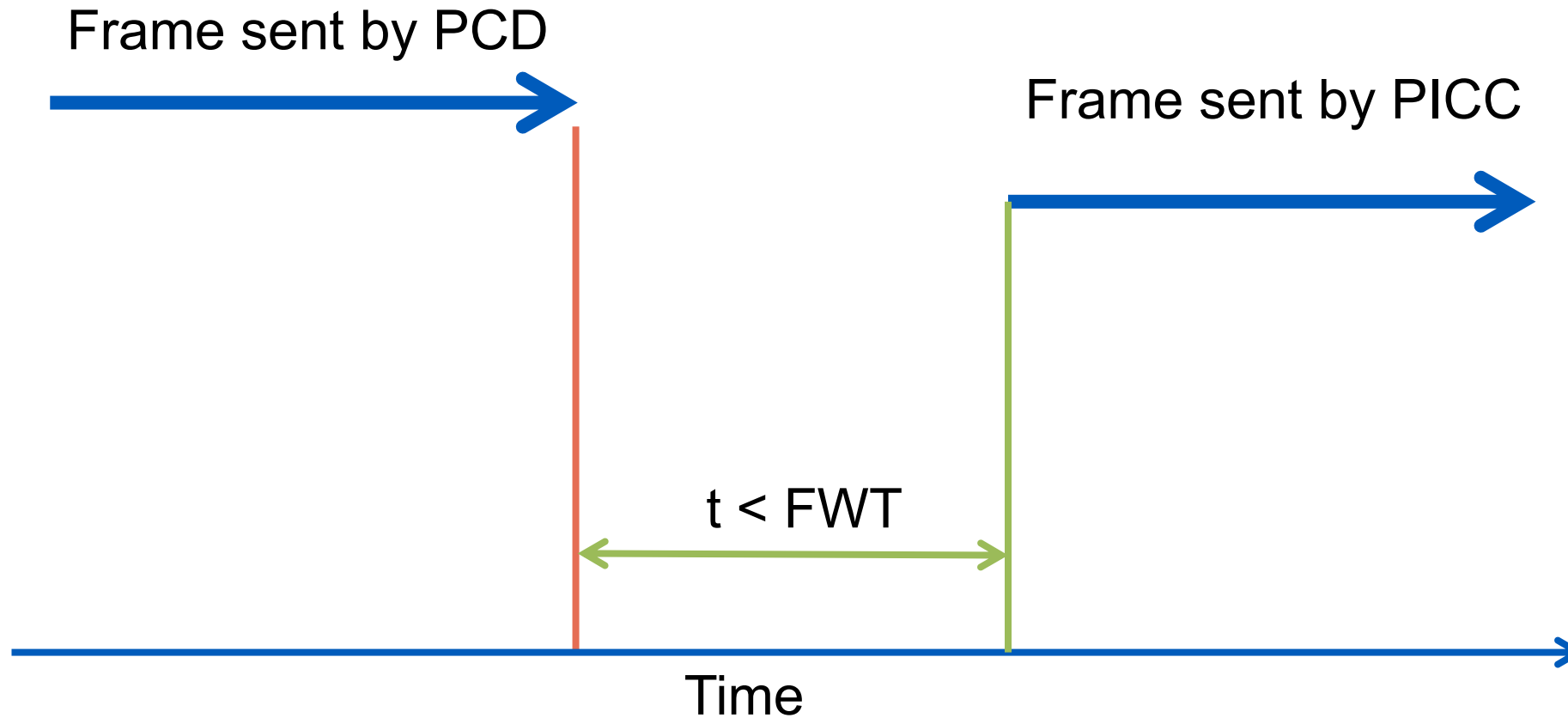
Relay Attack on NFC



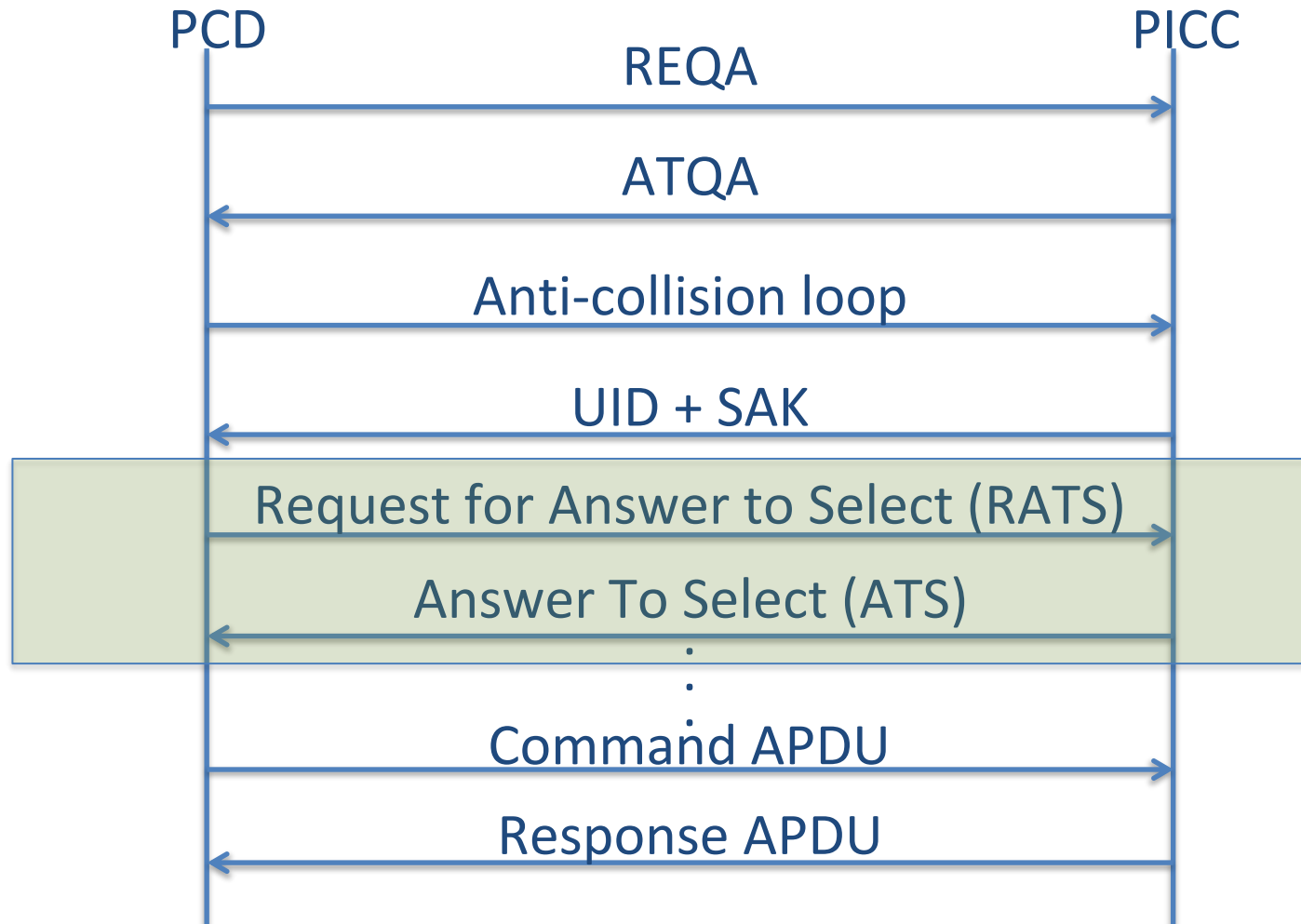
- Relaying e.g. contactless EMV payments from your pocket to a faraway shop
 - Requires card emulation on the proxy token
 - Does not require UID spoofing because EMV does not use the UID



Frame Waiting Time (FWT)



NFC reader and tag interaction



FWT parameter

Answer To Select (ATS)	
TL	Length Byte
T0	Format Byte
TA	Interface Bytes
TB	
TC	
T1	Historical Bytes (ISO/IEC 7816-4)
Tk	
CRC 1	
CRC 2	

TB	
FWI	SFGI

FWI: Frame Waiting Time Integer

SFGI: Start-up Frame Guard Time

$$FWT = (256 \times 16 / f_c) \times 2^{FWI}$$

$$FWT_{\text{Min}} = 0: (256 \times 16 / 13.56 \times 10^6) \times 1 \approx 303 \mu\text{s}$$

$$FWT = 4: (256 \times 16 / 13.56 \times 10^6) \times 2^4 \approx 4833 \mu\text{s}$$

$$FWT = 8: (256 \times 16 / 13.56 \times 10^6) \times 2^8 \approx 77 \text{ ms}$$

$$FWT_{\text{Max}} = 14: (256 \times 16 / 13.56 \times 10^6) \times 2^{14} \approx 4949 \text{ ms}$$

Observation on Android Jelly Bean (4.1.2)

- MIFARE DESFire:
 - default FWI = 0x8
 - FWT = 77 ms
- Nexus S responded well beyond 77 ms ($\approx 430\text{ms}$)
- Changing FWI parameter doesn't affect response time
- We assume fixed FWT implementation
- Readers often ignores FWT configurations

NFC on mobile phones

- Integration of NFC in mobile phone is growing
 - Nokia Lumia 610 and above
 - Nexus S and above
 - Samsung Galaxy S II plus and above
 - BlackBerry 10
 - iPhone 6 and above
 - Many more

NFC support on phones

- Mostly **Read/write** and **P2P** mode
- Some phone platform include **Secure Element** necessary for Card Emulation, **Host Card Emulation API** is available on Android 4.4 and above
- Currently used applications
 - NDEF tag read/write
 - FourSquare check-ins
 - Samsung TectTiles
 - Public transit tickets
 - Mobile payment
 - Loyalty card

Threats on NFC phones

- Denial of Service attacks

- Mobile phone NFC stack reacts to any tag within its NFC range
- Some mal-formatted tags can jam the stack
- Also, most of the card manager in Security Element (SE) blocks itself after 10 successive authentication failures

- Malware delivery via NFC

- Mobile OSs reads NDEF message and opens corresponding application
 - E.g. NDEF with URL causes phone to open the URL in its default web browser
- NDEF with URL to malware download page
- NFC message with malicious content
 - Malicious file over NFC to exploit android document viewer vulnerability
 - NFC to execute Unstructured Supplementary Service Data (USSD) codes

NFC based financial application on phone

- Rely on security offered by the **mobile phone platform**
 - Sandboxes
 - Permission based access control
- Should protect against **mobile malwares**
- **Ill intent** of the phone user
 - E.g. User may gain root access to modify ticket value
- Protect against remote and local attacker
- Ensure the security even when the **OS is compromised**



Secure execution on mobile phone

- **Isolate Execution**
 - Execution of a security-sensitive code in complete isolation from other codes
 - Ensures **integrity** and **run-time secrecy** of application data
- **Secure Storage**
 - Protects stored data from unauthorized access
 - Passwords, secret keys, credentials etc.
- **Remote Attestation**
 - Remotely **verify authenticity** of any particular application before interaction
 - Root of trust measurement
 - Dynamic root of trust measurement (DRTM)



Secure execution on mobile phone (Cont'd)

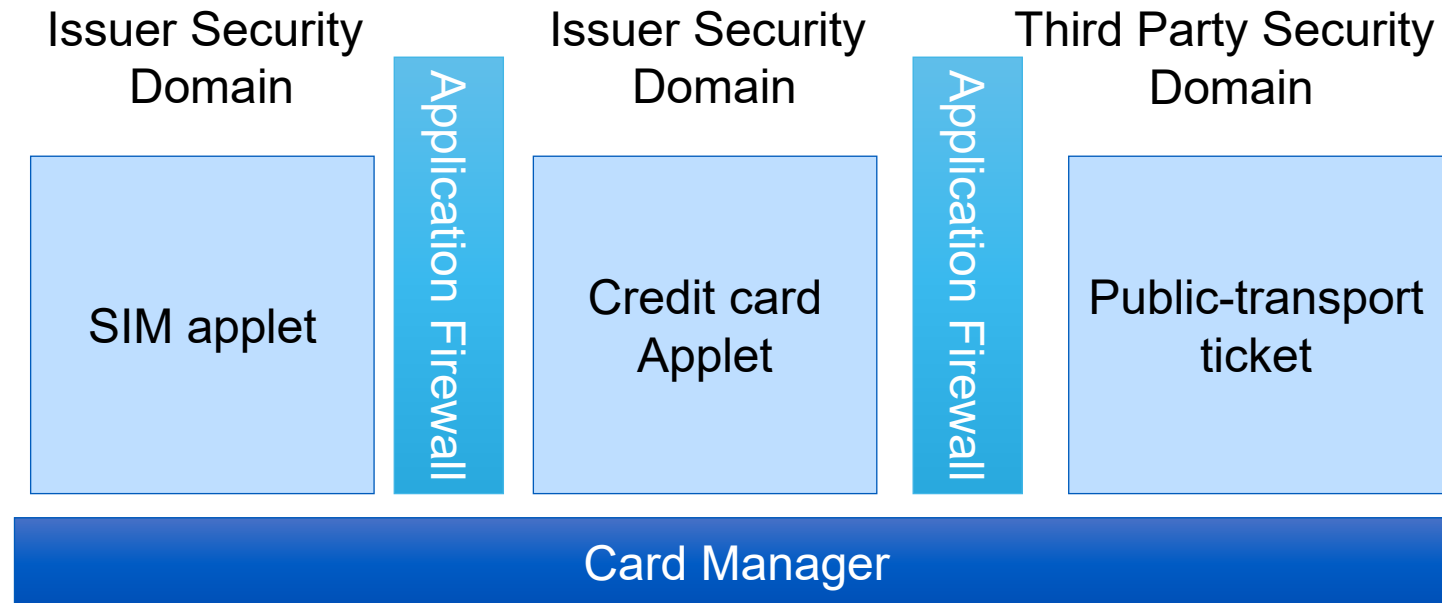
- **Secure Provisioning**
 - Securely deploying application module to a specific user device from a remote server over the air
 - Application migration from one device to another
- **Trusted path**
 - Ensures unaltered communication channel between the end points
 - Direct physical connection between NFC front end controller and the isolated execution environment



Available Secure Elements

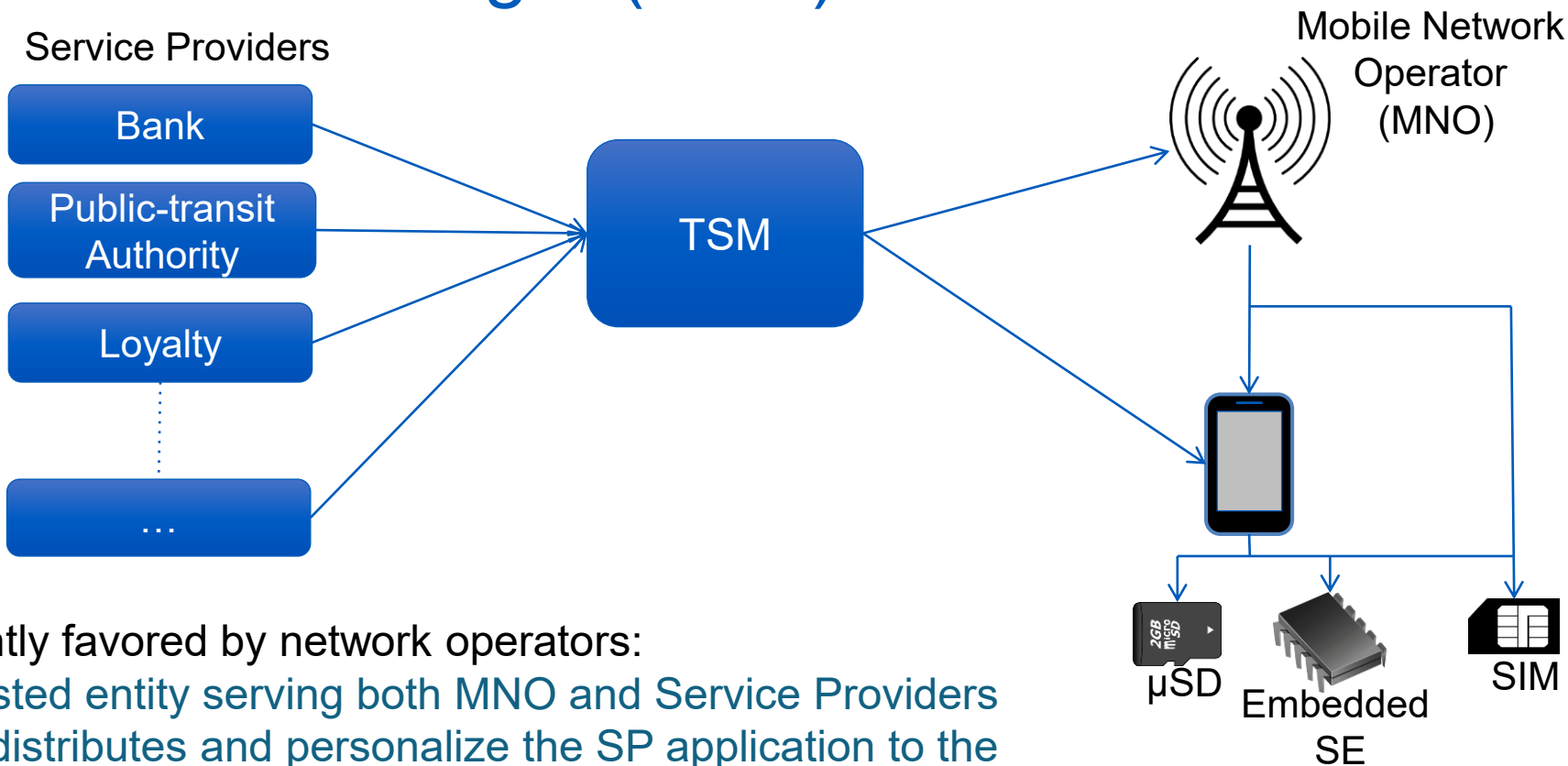
- Contactless stickers
 - Independent of the mobile phone OS
 - Elisa Lompakko
- Universal Integrated Circuit Card (UICC)
 - Preferred by Mobile Network Operators
 - Orange Quick Tap
- Secure MicroSD
 - Used by some banks in Taiwan
- Embedded Secure Element
 - Google Wallet
- Programmable Trusted Execution Environment
 - On-board Credential (ObC)
 - Mobicore

Security Element Architecture, e.g. SIM



- SIM is multi-application smart card
- Each service provider creates a separate Security Domain on the card
- Problems: increases the complexity of card manager; over-the-air installation of new applets is challenging

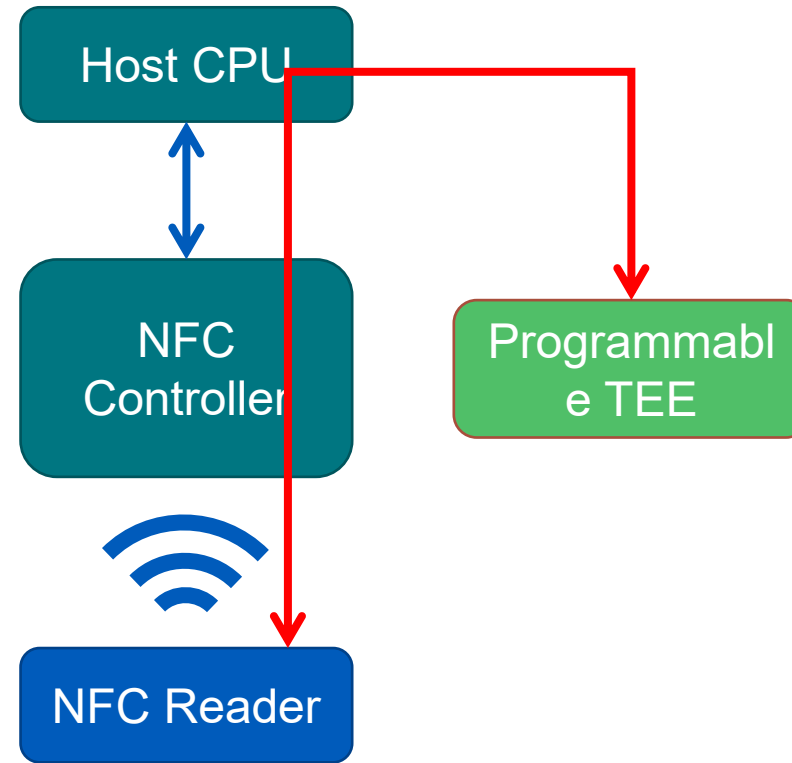
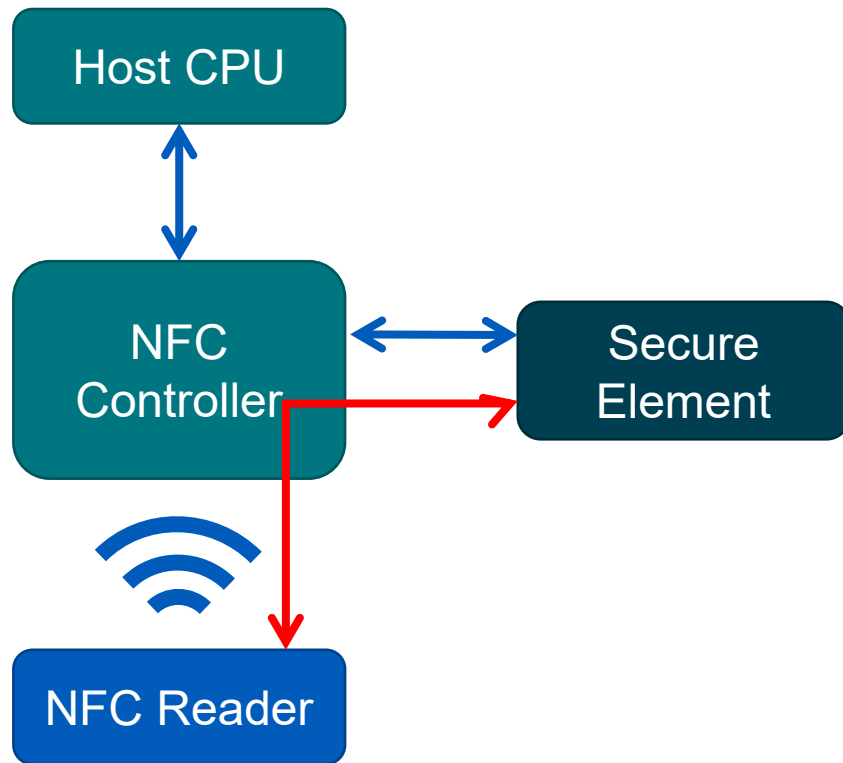
Trusted Service Manager (TSM)



Model currently favored by network operators:

- Single trusted entity serving both MNO and Service Providers
- Securely distributes and personalizes the SP application to the customer's SE over the air (OTA personalization).
- Verify user's device and SE capabilities and resources
- Manage life cycle of the applications

Host Card Emulation



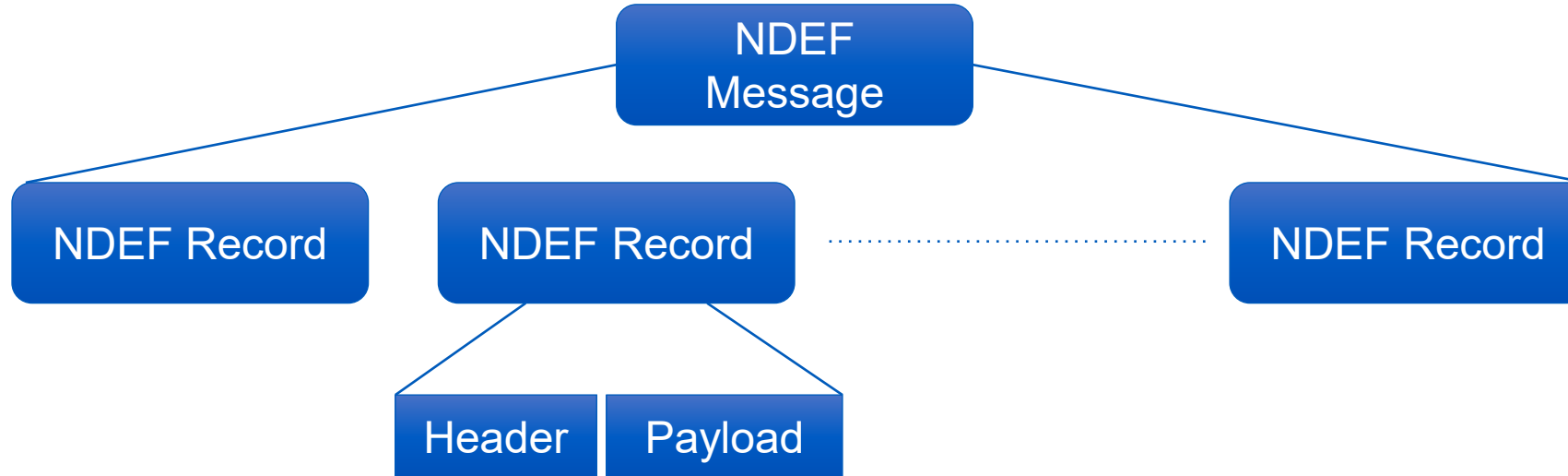
NFC card emulation with a secure element NFC card emulation with a Programmable TEE



NFC applications

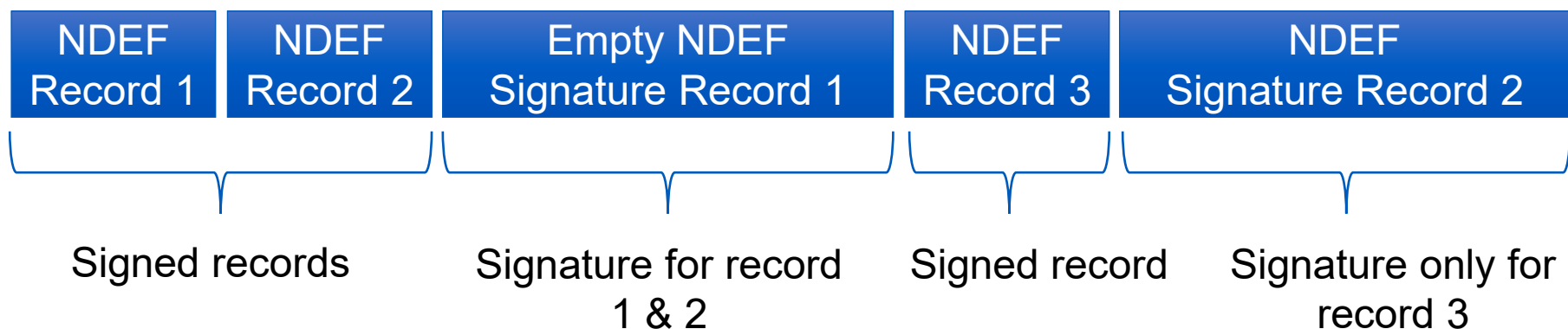


NFC Data Exchange Format (NDEF)



- NDEF is Message encapsulation format.
- Used to exchange messages between:
 - NFC devices or
 - An NFC device and a tag.
- Contains one or more NFC application data as NDEF Record
- Header defines the properties of the Payload.
 - Start and end of NDEF records
 - Record type definition (RTD): payload data type
 - Length of the payload etc.

Signature Record Type Definition



- Provides integrity and authenticity
- Signature RTD contains:
 - Signature,
 - RSA (1024) with SHA-1 and PKCS#1 v 1.5 padding or PSS
 - ECDSA (P-192) with SHA-1 with no padding.
 - Certificate chain.
 - Or, reference location to the signature
- Signature Record apply for
 - all preceding records, (from record 1) or,
 - Record following the preceding Signature record.
- Vulnerability
 - Cloning, replacing a tag with another valid tag

Tag UID based NFC applications

- Simple Access control application based on tag UID
 - NFC tags is used as credential to identify the user
 - Reader must be connected to a backend database
 - Backend server maintains access policies
- Pros:
 - Simple and cheap solution
 - “UID cannot be faked easily !!”
 - Suitable for small scale business
- Cons:
 - Backend complexity increases with the number of customers
 - Readers needs to be connected to the backend server all the time.

Event Ticketing

- One time or limited use tags
 - MIFARE Ultralight / Ultralight C
- Reader implements cryptographic functionality
 - **Key diversification** – e.g. Hash(UID + Master key)
 - **Encrypts** data and **store** the cipher-text on tags.
 - Reads the cipher-text from tags and decrypts data.
- Use of **OTP** bytes as **incremental counter**
- Use **Lock bytes** to prevent **rewrite**
- **MAC** for integrity protection
- Authentication keys for access control



Public transit application

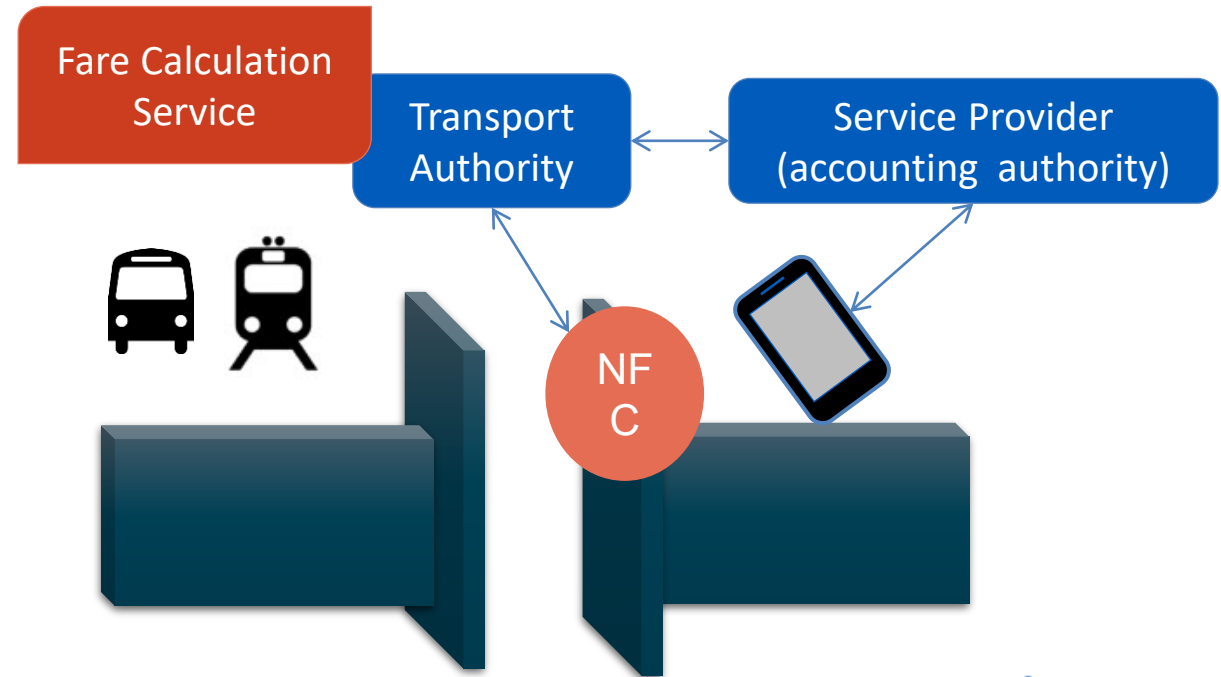
- Proprietary solutions are widely used
 - MIFARE Classic
 - MIFARE Ultralight/Ultralight C
 - MIFARE DESFire EV1
 - Uses Symmetric crypto
 - Triple-DES, AES
 - Value is stored on the card
- Standards
 - ITSO: Interoperable public transport ticketing using contactless smart customer media.

Open Payment Ticketing

- Smart Card Alliance proposal for NFC ticketing
- Each traveler has a travel account in a server cloud, which is operated by a service provider (SP)
- Travel card only stores user's identity and credentials
 1. User identity is verified by a reader at the station gate
 2. Ticket identity and travel information sent to a backend server
 3. The backend server calculates the ticket fare and forwards the information to SP for payment collection
 4. Payment is collected by SP
- Allows credentials from different SPs to be used
 - E.g. Bank cards, SIM card, National ID etc.

Open Payment Ticketing with Mobile Phone

- **Transport Authority**
 - Operates transport system
 - Calculate the fare calculation based on the ID and traveling distance
 - Collects ticketing evidence for auditing
- **Service provider** (e.g. bank or mobile operator)
 - Manages the customer relationship and travel account; issues the travel credentials
 - Collects evidence directly from phones and from Transport Authority for auditing
 - Collects payment from the customer (prepaid or credit)





Mobile Payment Security Analysis: Vulnerabilities of Google Wallet



PIN Storage Vulnerability

- PIN entry required for transactions
- Only six tries permitted
- But an attacker who steals a device and then roots it can extract the PIN from the salted hash
 - Because it's not stored on the SE
 - Storing it on the SE would make banks liable for breaches due to stolen PINs
 - Links Ch 9s, 9t

PIN Storage

- PIN is salted with a 64-bit random value and hashed with one round of SHA-256

```
protected boolean doSetPin(WalletClient.DeviceInfo.PinInfo.Builder
paramBuilder, UserPin paramUserPin)
{
    WLog.d(TAG, "doSetPin");
    int i = userPinAsInt(paramUserPin);
    long l = this.mSecureRandom.nextLong();
    String str = hashPin(i, l);
    paramBuilder.setPinTryCounter(6).setPinTryLimit(6).setLocalSalt(l)
.setLocalPinHash(str);
    return true;
}
...
String hashPin(int paramInt, long paramLong)
{
    String str = Integer.toString(paramInt) + Long.toString(paramLong);
    return this.mDigestUtil.sha256(str);
}
```

Storage of Hash

- Salt and hash stored in a SQLite database in Google Wallet's /data directory
 - `/data/data/com.google.android.apps.walletnfcrel/databases/walletDatastore`
- "Wallet Cracker" simply tries all 10,000 four-digit PINs to find PIN from the hash

Google's Response

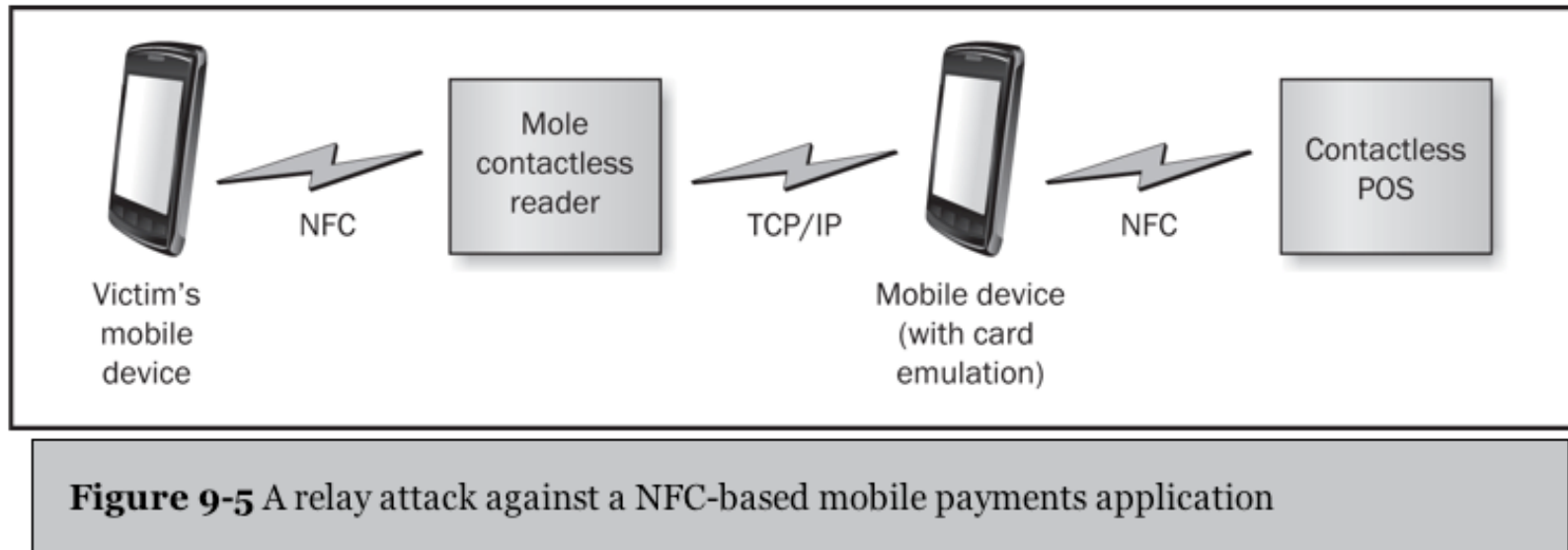
- Don't run Google Wallet on rooted phones
 - Not very reassuring since the thief can root your phone
- Much better to perform PIN storage and verification on the SE
 - Also store the PIN try counter on the SE

Countermeasures for Google Wallet Cracker

- Don't root your device
- Enable Android lock screen
- Disable ADB debugging
- Keep up-to-date with patches



Relay Attacks (MITM)



- "Mole" reader gets close to target mobile device
- Attacker's mobile gets near POS terminal
- APDUs are passed via TCP/IP

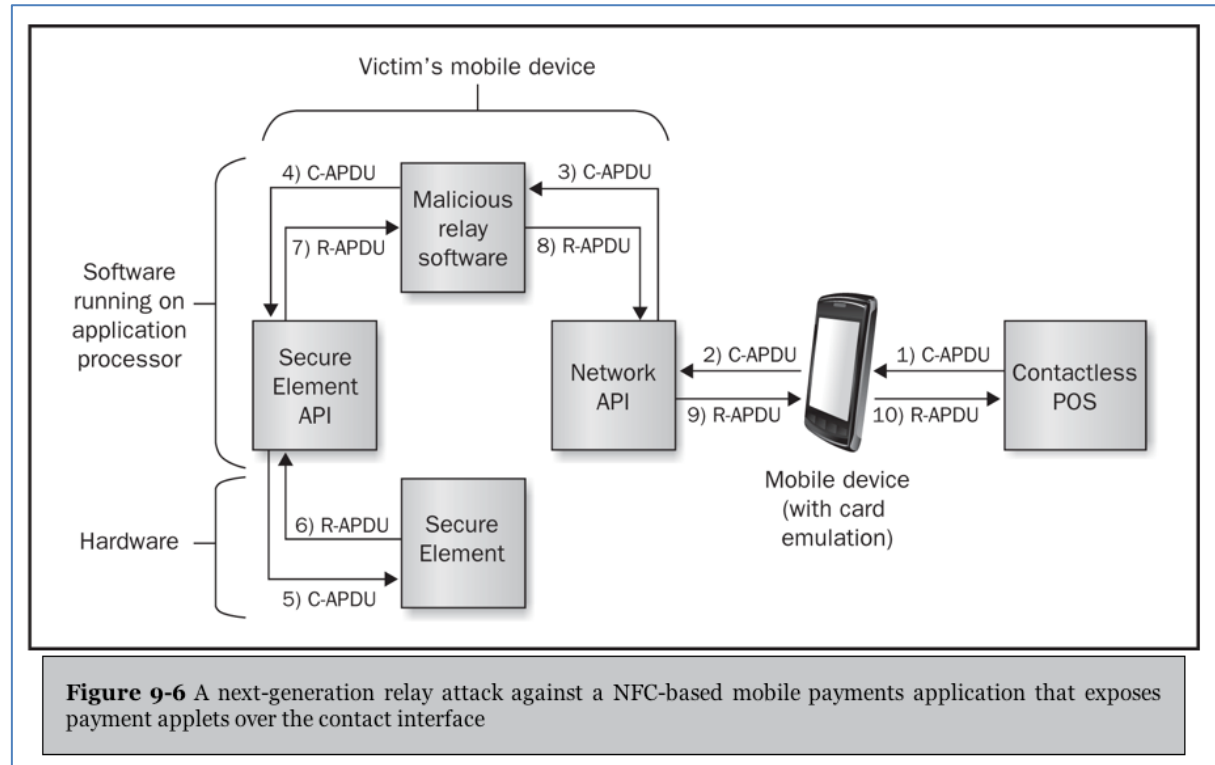
Relay Attack Limitations

- Target's mobile payment app must be unlocked
- Google Wallet requires entry of a PIN to unlock



Relay Through a Malicious App

- Works against Google Wallet
- Because it exposes payment credentials to the contact interface
- Requires root privileges to bypass SE API signature authentication



Relay Attack Countermeasures

- Contactless POS terminals should enforce a time-out on all transactions
 - Relay attack requires network communications which slows it down
 - Not very practical because errors can cause delays in legitimate transactions
- Use location information to flag suspicious transactions
 - Target mobile is not really near the POS
 - Requires target GPS to be active and consumer's consent



Relay Attack Countermeasures

- Google Wallet is no longer vulnerable to the second attack
 - It no longer exposes payment applets over the contact interface



Additional references

- NFC Data Exchange Format (NDEF) Technical Specification
- Madlmayr, G.; Langer, J.; Kantner, C.; Scharinger, J.; , "NFC Devices: Security and Privacy," Availability, Reliability and Security, 2008. ARES 08. Third International Conference on , vol., no., pp.642-647, 4-7 March 2008 doi: 10.1109/ARES.2008.105
- L. Francis, G. P. Hancke, K. E. Mayes, and K. Markantonakis. Practical Relay Attack on Contactless Transactions by Using NFC Mobile Phones. Cryptology ePrint Archive, Report 2011/618, 2011.
<http://eprint.iacr.org/2011/618>.

Acknowledgement

- Slides are adapted from talks given by:
 - Sandeep Tamrakar from Aalto University: “NFC Application Security”
 - Sam Bowne from City College of San Francisco: “Mobile Payment” in course CNIT 128: Hacking Mobile Devices